Parallelization of Diophantine Solvers

Hamza Iseric, Elijah Kin, Cameron Yeagle

May 5, 2026

Introduction

A *Diophantine* equation is an equation for which we are only interested in integer solutions.

Introduction

A *Diophantine* equation is an equation for which we are only interested in integer solutions. For instance, Pythagorean triples (e.g. $3^2 + 4^2 = 5^2$) are the solutions of the Diophantine equation $a^2 + b^2 = c^2$, while Fermat's last theorem famously states that $a^n + b^n = c^n$ has no solutions in the positive integers for $n \geq 3$.

Introduction

A *Diophantine* equation is an equation for which we are only interested in integer solutions. For instance, Pythagorean triples (e.g. $3^2 + 4^2 = 5^2$) are the solutions of the Diophantine equation $a^2 + b^2 = c^2$, while Fermat's last theorem famously states that $a^n + b^n = c^n$ has no solutions in the positive integers for $n \geq 3$.

Question: Does

$$a_1^6 + a_2^6 = b_1^6 + b_2^6 + b_3^6 + b_4^6 + b_5^6$$

have any solutions in the positive integers?

Introduction

A *Diophantine* equation is an equation for which we are only interested in integer solutions. For instance, Pythagorean triples (e.g. $3^2 + 4^2 = 5^2$) are the solutions of the Diophantine equation $a^2 + b^2 = c^2$, while Fermat's last theorem famously states that $a^n + b^n = c^n$ has no solutions in the positive integers for $n \geq 3$.

Question: Does

$$a_1^6 + a_2^6 = b_1^6 + b_2^6 + b_3^6 + b_4^6 + b_5^6$$

have any solutions in the positive integers?

Answer: Yes! The smallest was found in 2002:

$$1117^6 + 770^6 = 1092^6 + 861^6 + 602^6 + 212^6 + 84^6$$

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9).

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7.

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2.

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2. Since there are only five terms on the right-hand side with each congruent to either 0 or 1, this means that at least three of the b_i are divisible by 7.

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2. Since there are only five terms on the right-hand side with each congruent to either 0 or 1, this means that at least three of the b_i are divisible by 7. This yields the helpful restriction

$$b_2^6 \equiv a_1^6 + a_2^6 - b_1^6 \mod 7^6.$$

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2. Since there are only five terms on the right-hand side with each congruent to either 0 or 1, this means that at least three of the b_i are divisible by 7. This yields the helpful restriction

$$b_2^6 \equiv a_1^6 + a_2^6 - b_1^6 \mod 7^6.$$

Finally, we define $v = a_1^6 + a_2^6 - (b_1^6 + b_2^6)$ and check if we can decompose $v/7^6 = c_1^6 + c_2^6 + c_3^6$.

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2. Since there are only five terms on the right-hand side with each congruent to either 0 or 1, this means that at least three of the b_i are divisible by 7. This yields the helpful restriction

$$b_2^6 \equiv a_1^6 + a_2^6 - b_1^6 \mod 7^6.$$

Finally, we define $v = a_1^6 + a_2^6 - (b_1^6 + b_2^6)$ and check if we can decompose $v/7^6 = c_1^6 + c_2^6 + c_3^6$. Modular restrictions make this step tractable,

Resta-Meyrignac Algorithm

The crux of the search was that sixth powers mod 7 can be congruent only to either 0 or 1 (and likewise for mod 8 and mod 9). Then, if we are only interested in primitive solutions, at least one of a_1 and a_2 is not divisible by 7. Hence, mod 7 the left-hand side is either 1 or 2. Since there are only five terms on the right-hand side with each congruent to either 0 or 1, this means that at least three of the b_i are divisible by 7. This yields the helpful restriction

$$b_2^6 \equiv a_1^6 + a_2^6 - b_1^6 \mod 7^6.$$

Finally, we define $v = a_1^6 + a_2^6 - (b_1^6 + b_2^6)$ and check if we can decompose $v/7^6 = c_1^6 + c_2^6 + c_3^6$. Modular restrictions make this step tractable, i.e. it is impossible for a sum of three sixths powers to be congruent to $15 \mod 31$.

OpenMP Implementation

Our OpenMP program works by doing a triangular search over $1 \leq a_2 \leq a_1 \leq$ `a_max`.

OpenMP Implementation

Our OpenMP program works by doing a triangular search over $1 \leq a_2 \leq a_1 \leq$ `a_max`. Since each iteration of the a_1 loop is independent, it is ripe for parallelization!

OpenMP Implementation

Our OpenMP program works by doing a triangular search over $1 \leq a_2 \leq a_1 \leq$ a_max. Since each iteration of the a_1 loop is independent, it is ripe for parallelization! Using this program with a_max equal to 3000, we find the first several solutions in about 5 seconds with 32 threads on one Zaratan node:

OpenMP Implementation

Our OpenMP program works by doing a triangular search over $1 \leq a_2 \leq a_1 \leq$ a_max. Since each iteration of the a_1 loop is independent, it is ripe for parallelization! Using this program with a_max equal to 3000, we find the first several solutions in about 5 seconds with 32 threads on one Zaratan node:

$$1117^6 + 770^6 = 602^6 + 212^6 + 1092^6 + 861^6 + 84^6$$
$$2041^6 + 691^6 = 1893^6 + 1468^6 + 1407^6 + 1302^6 + 1246^6$$
$$2441^6 + 752^6 = 2096^6 + 1266^6 + 2184^6 + 1484^6 + 1239^6$$
$$2827^6 + 151^6 = 1488^6 + 390^6 + 2653^6 + 2296^6 + 1281^6$$
$$2959^6 + 2470^6 = 2481^6 + 850^6 + 2954^6 + 798^6 + 420^6$$

OpenMP Implementation

Our OpenMP program works by doing a triangular search over $1 \leq a_2 \leq a_1 \leq$ a_max. Since each iteration of the a_1 loop is independent, it is ripe for parallelization! Using this program with a_max equal to 3000, we find the first several solutions in about 5 seconds with 32 threads on one Zaratan node:

$$1117^6 + 770^6 = 602^6 + 212^6 + 1092^6 + 861^6 + 84^6$$
$$2041^6 + 691^6 = 1893^6 + 1468^6 + 1407^6 + 1302^6 + 1246^6$$
$$2441^6 + 752^6 = 2096^6 + 1266^6 + 2184^6 + 1484^6 + 1239^6$$
$$2827^6 + 151^6 = 1488^6 + 390^6 + 2653^6 + 2296^6 + 1281^6$$
$$2959^6 + 2470^6 = 2481^6 + 850^6 + 2954^6 + 798^6 + 420^6$$

For comparison, the 2002 paper mentions it took about 5 minutes to obtain just the single smallest solution.

CUDA Implementation: Host Setup

The CUDA program keeps the Resta-Meyrignac search, but moves the expensive candidate testing to the GPU.

CUDA Implementation: Host Setup

The CUDA program keeps the Resta-Meyrignac search, but moves the expensive candidate testing to the GPU.

Before launching the kernel, the CPU builds two lookup tables:

- a residue table modulo $7^6 = 117649$ for possible b_2 values;
- a sorted table of pair sums $x^6 + y^6$ used later to recognize two of the remaining three terms.

CUDA Implementation: Host Setup

The CUDA program keeps the Resta-Meyrignac search, but moves the expensive candidate testing to the GPU.

Before launching the kernel, the CPU builds two lookup tables:

- a residue table modulo $7^6 = 117649$ for possible b_2 values;
- a sorted table of pair sums $x^6 + y^6$ used later to recognize two of the remaining three terms.

These tables are copied once to GPU memory with `cudaMemcpy`, so all GPU threads can reuse the same precomputed data.

CUDA Implementation: Parallel Work Split

The main search space is the triangular set

$$1 \leq a_2 \leq a_1 \leq a_{\max}.$$

CUDA Implementation: Parallel Work Split

The main search space is the triangular set

$$1 \leq a_2 \leq a_1 \leq a_{\max}.$$

The kernel assigns each CUDA thread a linear index into this triangular space, then converts it back to a pair (a_1, a_2) using triangular numbers.

CUDA Implementation: Parallel Work Split

The main search space is the triangular set

$$1 \leq a_2 \leq a_1 \leq a_{\max}.$$

The kernel assigns each CUDA thread a linear index into this triangular space, then converts it back to a pair (a_1, a_2) using triangular numbers.

Each thread then advances by a grid-wide stride:

$$\text{pair\_index} = \text{thread id},\ \text{thread id} + \text{grid size}, \ldots$$

CUDA Implementation: Parallel Work Split

The main search space is the triangular set

$$1 \leq a_2 \leq a_1 \leq a_{\max}.$$

The kernel assigns each CUDA thread a linear index into this triangular space, then converts it back to a pair (a_1, a_2) using triangular numbers.

Each thread then advances by a grid-wide stride:

$$\text{pair\_index} = \text{thread id},\ \text{thread id} + \text{grid size}, \ldots$$

This gives a simple load distribution without storing all (a_1, a_2) pairs explicitly.

CUDA Implementation: Filtering Candidates

For each assigned (a_1, a_2), the kernel computes

$$L = a_1^6 + a_2^6$$

using 128-bit integer arithmetic.

CUDA Implementation: Filtering Candidates

For each assigned (a_1, a_2), the kernel computes

$$L = a_1^6 + a_2^6$$

using 128-bit integer arithmetic.
It first skips non-primitive candidates where both a_i are divisible by 2 or both are divisible by 3.

CUDA Implementation: Filtering Candidates

For each assigned (a_1, a_2), the kernel computes

$$L = a_1^6 + a_2^6$$

using 128-bit integer arithmetic.
It first skips non-primitive candidates where both a_i are divisible by 2 or both are divisible by 3.

Then it loops over possible b_1 values and uses the modulo 7^6 table to find only compatible b_2 residues:

$$b_2^6 \equiv L - b_1^6 \pmod{7^6}.$$

CUDA Implementation: Filtering Candidates

For each assigned (a_1, a_2), the kernel computes

$$L = a_1^6 + a_2^6$$

using 128-bit integer arithmetic.
It first skips non-primitive candidates where both a_i are divisible by 2 or both are divisible by 3.

Then it loops over possible b_1 values and uses the modulo 7^6 table to find only compatible b_2 residues:

$$b_2^6 \equiv L - b_1^6 \pmod{7^6}.$$

Additional modular filters modulo $13, 19, 27, 31, 32, 49$ remove values that cannot be written as a sum of three sixth powers.

CUDA Implementation: Final Decomposition

After choosing b_1 and b_2, the remaining value is

$$v/7^6 = \frac{a_1^6 + a_2^6 - b_1^6 - b_2^6}{7^6}.$$

CUDA Implementation: Final Decomposition

After choosing b_1 and b_2, the remaining value is

$$v/7^6 = \frac{a_1^6 + a_2^6 - b_1^6 - b_2^6}{7^6}.$$

The device function `try_decompose` checks whether

$$v/7^6 = c_1^6 + c_2^6 + c_3^6.$$

CUDA Implementation: Final Decomposition

After choosing b_1 and b_2, the remaining value is

$$v/7^6 = \frac{a_1^6 + a_2^6 - b_1^6 - b_2^6}{7^6}.$$

The device function `try_decompose` checks whether

$$v/7^6 = c_1^6 + c_2^6 + c_3^6.$$

It loops over c_3, applies more modular impossibility filters, and then binary-searches the sorted pair table for

$$c_1^6 + c_2^6 = v/7^6 - c_3^6.$$

CUDA Implementation: Final Decomposition

After choosing b_1 and b_2, the remaining value is

$$v/7^6 = \frac{a_1^6 + a_2^6 - b_1^6 - b_2^6}{7^6}.$$

The device function `try_decompose` checks whether

$$v/7^6 = c_1^6 + c_2^6 + c_3^6.$$

It loops over c_3, applies more modular impossibility filters, and then binary-searches the sorted pair table for

$$c_1^6 + c_2^6 = v/7^6 - c_3^6.$$

If this succeeds, the stored solution is $(a_1, a_2, b_1, b_2, 7c_1, 7c_2, 7c_3)$.

CUDA Implementation: Collecting Results

Multiple GPU threads may find solutions at the same time, so the kernel uses `atomicAdd` to reserve an output slot safely.

CUDA Implementation: Collecting Results

Multiple GPU threads may find solutions at the same time, so the kernel uses `atomicAdd` to reserve an output slot safely.

After the kernel finishes:

- the host copies the solution count and stored solutions back;
- solutions are sorted before printing;
- the program reports total runtime and GPU device information.

CUDA Implementation: Collecting Results

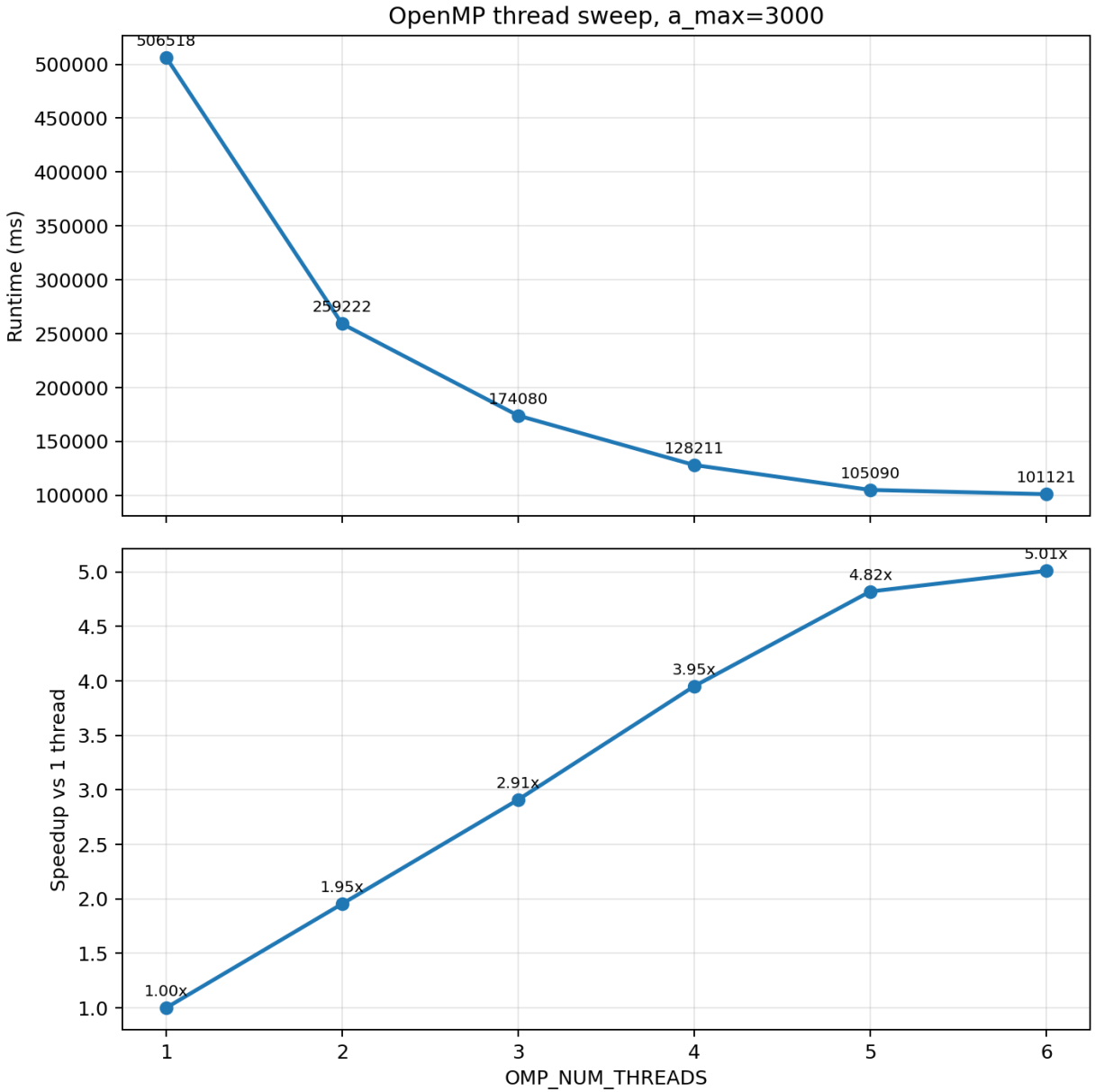
Multiple GPU threads may find solutions at the same time, so the kernel uses `atomicAdd` to reserve an output slot safely.

After the kernel finishes:

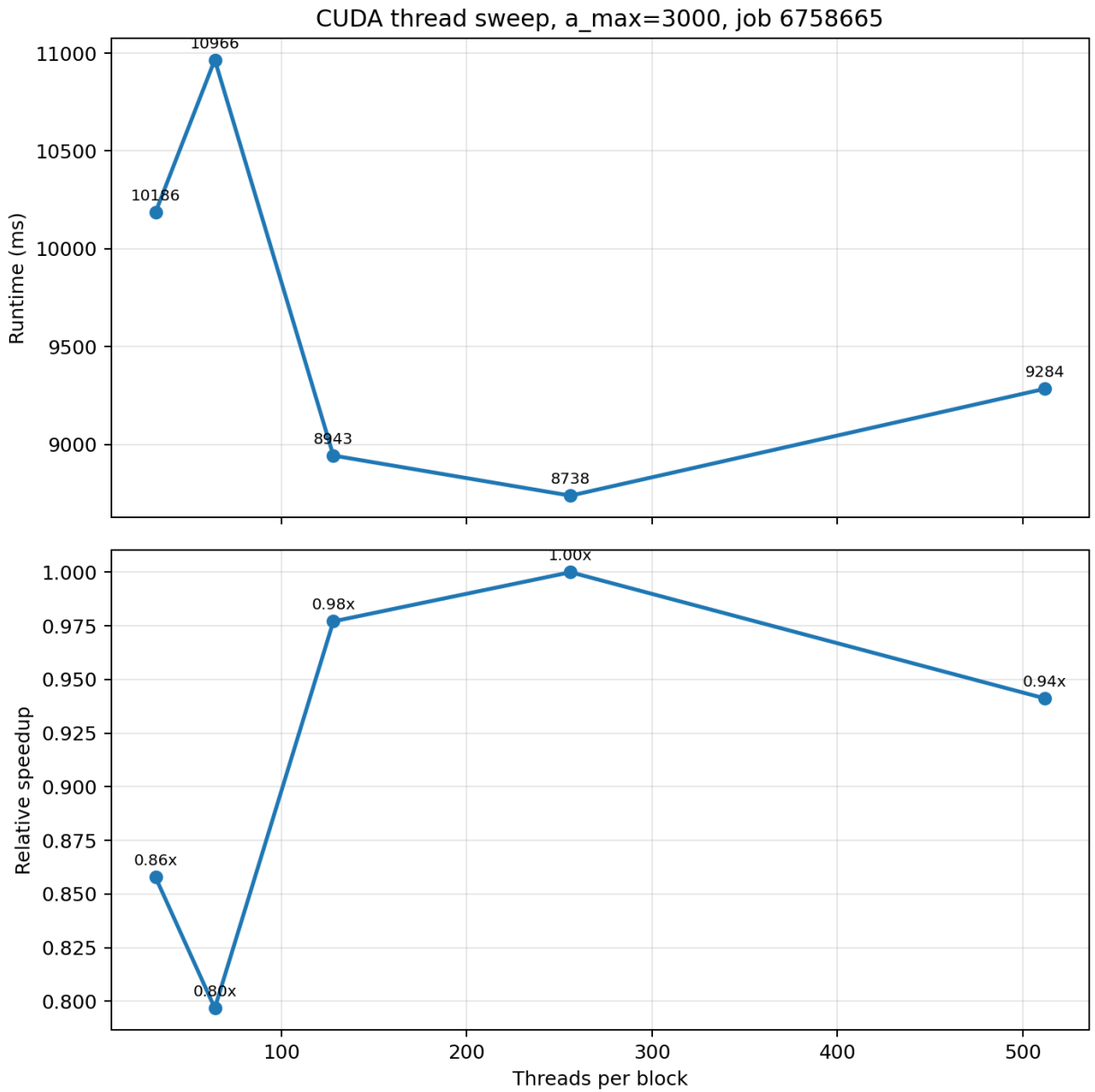
- the host copies the solution count and stored solutions back;
- solutions are sorted before printing;
- the program reports total runtime and GPU device information.

In short, CUDA parallelizes the outer (a_1, a_2) search, while modular arithmetic and precomputed tables keep each thread's inner search small.

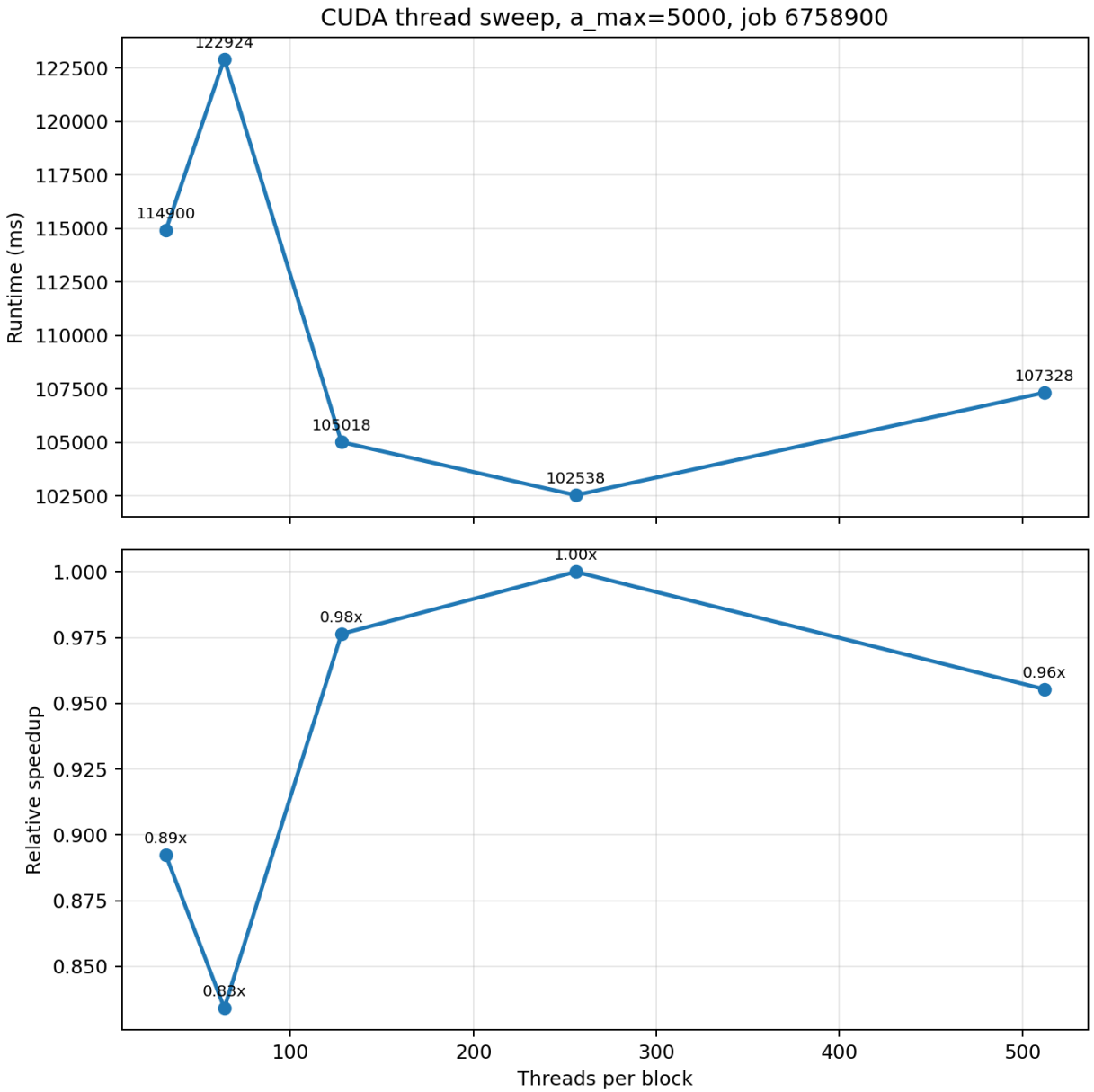
Performance: OpenMP



Performance: CUDA at $a_{\max} = 3000$



Performance: CUDA at $a_{\max} = 5000$



Performance Takeaways

- OpenMP shows strong thread scaling for this search on the CPU.
- CUDA gives lower absolute runtime, but the current kernel does not scale strongly with threads per block.
- The bottleneck is implementation quality of the GPU kernel, not the absence of parallel work in the problem itself.

Generalizability

New Equation:

$$a_1^6 = b_1^6 + b_2^6 + b_3^6 + b_4^6 + b_5^6 + b_6^6$$

Generalizability

New Equation:

$$a_1^6 = b_1^6 + b_2^6 + b_3^6 + b_4^6 + b_5^6 + b_6^6$$

Key Changes: 1 LHS term means a_1^6 is strictly congruent to 1 modulo 7.

Generalizability

New Equation:

$$a_1^6 = b_1^6 + b_2^6 + b_3^6 + b_4^6 + b_5^6 + b_6^6$$

Key Changes: 1 LHS term means a_1^6 is strictly congruent to 1 modulo 7. This implies that only 1 RHS term is 1 modulo 7, rest are 0. Hence,

$$a_1^6 \equiv b_1^6 \pmod{7^6}$$

avoiding $O(N)$ search for b_1.

Performance

# Threads	1	2	4	8	16	32	64	128
2 LHS, 5 RHS (ms)	136,298	71,504	36,329	18,402	9,186	4,842	2,618	1,466
1 LHS, 6 RHS (ms)	127,541	67,311	47,525	29,703	20,703	11,160	9865	10745

Figure: Execution time scaling: 2 LHS, 5 RHS ($a_{max} = 3000$) vs. 1 LHS, 6 RHS ($a_{max} = 6250$).

Performance

# Threads	1	2	4	8	16	32	64	128
2 LHS, 5 RHS (ms)	136,298	71,504	36,329	18,402	9,186	4,842	2,618	1,466
1 LHS, 6 RHS (ms)	127,541	67,311	47,525	29,703	20,703	11,160	9865	10745

Figure: Execution time scaling: 2 LHS, 5 RHS ($a_{max} = 3000$) vs. 1 LHS, 6 RHS ($a_{max} = 6250$).

Difficulty: **Scalability**

- Inner loop must resolve 5 unknown variables

No solutions found for $a_1 < 730,000$ (J.-C. Meyrignac, et al.).

References

G. Resta and J.-C. Meyrignac.
The smallest solutions to the diophantine equation $x^6 + y^6 = a^6 + b^6 + c^6 + d^6 + e^6$.
Math. Comput., 72(242):1051–1054, Apr. 2003.